

LAB TASK#4

NAME: MASHAAL ISHTIAQ

ROLL#: 22P-9326

SECTION: CS-4C

CODE:

```
#include <iostream>
#include <string>
using namespace std;

class StudentRecord {
private:
int id;
string name;
float gpa;

public:
// Constructor
StudentRecord(int id, string name, float gpa) {
this->id = id;
this->name = name;
this->gpa = gpa;
}

// Getter functions to access the private data members
int getId() {
return id;
}
string getName() {
return name;
}
float getGpa() {
return gpa;
}

// Function to print
void printDetails() {
cout << "ID: " << id << ", Name: " << name << ", GPA: " << gpa << endl;
}
};

struct Node {
StudentRecord data;
Node* next;

// Constructor to initialize local data members
Node(int id, string name, float gpa) : data(id, name, gpa), next(nullptr) {}
};

class Link_List {
private:
Node* head;
```

```
float total_gpa;
int length;

public:
Link_List() {
head = nullptr;
total_gpa = 0;
length = 0;
}

~Link_List() {
while (head != nullptr) {
Node* temp = head;
head = head->next;
delete temp;
}
}

// Function to add a student
void add_std(int id, string name, float gpa) {
Node* newNode = new Node(id, name, gpa);
newNode->next = head;
head = newNode;
total_gpa += gpa;
length++;
cout << "Student added successfully" << endl;
}

// Function to remove a student by ID
void remove_id(int id) {
Node* curr = head;
Node* slow = nullptr;

while (curr != nullptr && curr->data.getId()!= id) {
slow = curr;
curr = curr->next;
}

if (curr == nullptr) {
cout << "No student record available" << endl;
return;
}

if (slow == nullptr) {
head = curr->next;

} else {
slow->next = curr->next;
}

total_gpa -= curr->data.getGpa();
delete curr;
```

```
cout << "Student removed successfully" << endl;
}

// Function to display all records
void display_records() {
if (head == nullptr) {
cout << "No student records available" << endl;
return;
}

cout << "Student Records:" << endl;
Node* curr = head;
while (curr != nullptr) {
curr->data.printDetails();
curr = curr->next;
}
}

// Function to search for a student by ID and display their information
void search_std_byID(int id) {
Node* temp = head;
while (temp != nullptr) {
if (temp->data.getId() == id) {
cout << "Student found:" << endl;
temp->data.printDetails();
return;
}
temp = temp->next;
}
cout << "Student with ID " << id << " not found" << endl;
}

// Function to calculate the average GPA of all students
float calc_average_gpa() {
if (length == 0)
return 0;
return total_gpa / length;
}
};

int main() {
Link_List std_List;

cout << "_____Welcome to the Student Registration System!_____" << endl;

while (1) {
cout << "\n1. Add a student\n";
cout << "2. Remove a student\n";
cout << "3. Search for a student\n";
cout << "4. Display all students\n";
cout << "5. Calculate average GPA\n";
cout << "6. Exit\n";
```

```
int choice;
cout << "Enter your choice: ";
cin >> choice;

switch (choice) {
case 1: {
int id;
string name;
float gpa;
cout << "Enter student ID: ";
cin >> id;
cout << "Enter student name: ";
cin >> name;
cout << "Enter student GPA: ";
cin >> gpa;
std_List.add_std(id, name, gpa);
break;
}
case 2: {
int id;
cout << "Enter student ID to remove: ";
cin >> id;
std_List.remove_id(id);
break;
}
case 3: {
int id;
cout << "Enter student ID to search: ";
cin >> id;
std_List.search_std_byID(id);
break;
}
case 4: {
std_List.display_records();
break;
}
case 5: {
cout << "Average GPA: " << std_List.calc_average_gpa() << endl;
break;
}
case 6: {
cout << "Goodbye!" << endl;
return 0;
}
default: {
cout << "Invalid choice. Try again." << endl;
}
}
}

return 0;
```

}

```
______Welcome to the Student Registration System!________

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 1
Enter student ID: 1001
Enter student name: john
Enter student GPA: 3.8
Student added successfully

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 1
Enter student ID: 1002
Enter student name: jane
Enter student GPA: 3.5
```

```
Student added successfully

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 1
Enter student ID: 1003
Enter student name: alice
Enter student GPA: 3.9
Student added successfully

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 4
Student Records:
ID: 1003, Name: alice, GPA: 3.9
ID: 1002, Name: jane, GPA: 3.5
ID: 1001, Name: john, GPA: 3.8
```

```
6. Exit
Enter your choice: 3
Enter student ID to search: 1002
Student found:
ID: 1002, Name: jane, GPA: 3.5

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 2
Enter student ID to remove: 1002
Student removed successfully

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 4
Student Records:
ID: 1003, Name: alice, GPA: 3.9
ID: 1001, Name: john, GPA: 3.8
```

```
ID: 1001, Name: John, GPA: 3.8

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 5
Average GPA: 2.56667

1. Add a student
2. Remove a student
3. Search for a student
4. Display all students
5. Calculate average GPA
6. Exit
Enter your choice: 6
Goodbye!
mashaal@mashaal-Latitude-7480:~/Desktop/semster_4
```